

this keyword

- ❖ 'this' is a keyword which act as a reference variable.
- ❖ 'this' reference variable contains address of current object.
- ❖ 'this' is an instance reference variable and cannot be accessed from static context.
- ❖ 'this' keyword can be used in three way

Consider the Following Hello class

```
class Hello {  
    int a;  
    int b;  
    void m1() {...}  
    void m1() {...}  
    Hello (int a, int b) {...}  
    Hello (int a) {...}  
    Hello () {...}  
}
```

1. To access the variable

Syntax:
this.<variable Name>

Example:

```
this. a;  
this. b;
```

2. To access the Methods

Syntax:
this.<Method Name>

Example:

```
this. m1();  
this. m2();
```

3. To access the variable

Syntax:
this.<Method Name>

Example:

```
this ();    //invoking Default Constructors  
this (99);  //invoking 1-Args Constructors  
this(99,88); //invoking 2-Args Constructors
```

- ❖ You can use same name for local variables and instance variable.
- ❖ When you access any variable directly then following thing will happen.
 - ⇒ Check whether that variable is declared in the local scope or not
 - ⇒ If found in the local scope, that local variable will be used.
 - ⇒ If not found in the local scope then checks whether that variable is declared in the class scope or not.
 - ⇒ If found that class level variable will be used.
- ❖ When you have same name for local variable and class level variables then do the following
 - ⇒ Refer the local variable directly.
 - ⇒ Refer the class level variable using this keyword.
- ❖ "this" is an instance reference variable and cannot be referenced from static context.
- ❖ Local variable cannot be referred using "this" keyword.
- ❖ The process of invoking one constructor from another constructor using this is called as CONSTRUCTOR CHAINING.
- ❖ "this" is a final reference variable and cannot be modified.
 - a. this = null; //INVALID
 - b. this = new Hello(); //INVALID

- ❖ Call to constructor using “this” must be from constructor only, not from methods & blocks.

```
class Hello {
    Hello () {}
    Hello (int a) {
        this ();           //VALID
    }
    {
        this ();           //INVALID
    }
    void show () {
        this ();           //INVALID
    }
}
```

- ❖ “this” can be assigned to current class reference variable.

<pre>class Hello { void show () { Hello h1 = this; //VALID Hai h2 = this; //INVALID } }</pre>	<pre>class Hai { void show () { Hello h1 = this; //INVALID Hai h2 = this; //VALID } }</pre>
---	---

- ❖ “this” keyword can be referred with current class name.

```
class Hello {
    int a;
    void show () {
        System.out.println (Hello. this. a); //VALID
        System.out.println (Hai. this. a);   //INVALID
    }
}
class Hai{
    int a;
}
```

Programs:

OOPS71.java

```
class OOPS71{
    public static void main(String args[]){
        Hello h=new Hello();
        h. show();
    }
}
class Hello{
    int a=10;
    void show(){
        System.out.println(a);
    }
}
```

OOPS72.java

```
class OOPS72{
    public static void main(String args[]){
        Hello h=new Hello();
        h. show();
    }
}
class Hello{
    int a=10;
    void show(){
        String a="India";
        System.out.println(a);
        System.out.println(a);
    }
}
```

OOPS73.java

```

class OOPS73{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show();
    }
    class Hello{
        int a=10;
        void show(){
            String a="India";
            System.out.println(a);
            System.out.println(this. a);
        }
    }
}

```

OOPS74.java

```

class OOPS74{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show();
    }
    class Hello{
        int a=10;
        static int b=20;
        void show(){
            String a="India";
            String b="Sai";
            System.out.println(a);
            System.out.println(b);
            System.out.println(this. a);
            System.out.println(this. b);
        }
    }
}

```

OOPS75.java

```

class OOPS75{
    public static void main(String args[]){
        Student stu1=new Student(88,"Sai");
        stu1.show();
        Student stu2=new Student(99,"Ram");
        stu2.show();
    }
    class Student {
        int sid;
        String sname;
        Student(int sid, String sname) {
            System.out.println("Stud 2-Arg Cons");
            sid = sid;
            sname = sname;
        }
        void show() {
            System.out.println(sid+"\t"+sname);
        }
    }
}

```

OOPS76.java

```

class OOPS76{
    public static void main(String args[]){
        Student stu1=new Student(88,"Ram");
        stu1.show();
        Student stu2=new Student(99,"Sai");
        stu2.show();
    }
    class Student {
        int sid;
        String sname;
        Student(int sid, String sname) {
            System.out.println("Stud 2-Arg Cons");
            this.sid=sid;
            this. sname=sname;
        }
        void show () {
            System.out.println(sid+"\t"+sname);
        }
    }
}

```

OOPS77.java

```

class OOPS77{
    public static void main(String args[]){
        Hello h1=new Hello();
        h1.show();
        Hello h2=new Hello(99);
        h2.show();
    }
    class Hello {
        int a;

```

```

        Hello() {
            System.out.println("Hello Def Cons");
        }
        Hello(int a) {
            System.out.println("hello 1-arg Cons");
            this. a=a;
        }
        void show() {
            System.out.println(a);
        }
    }
}

```

OOPS78.java

```
class OOPS78{
    public static void main(String args[]){
        Hello h=new Hello(99);
        h.show();
    }
    class Hello {
        int a;
        Hello(){
            System.out.println("Hello Defl Cons");
        }
        Hello(int a) {
            System.out.println("hello 1 arg Cons");
            this.a=a;
            Hello();
        }
        void show() {
            System.out.println(a);
        }
    }
}
```

OOPS79.java

```
class OOPS79{
    public static void main(String args[]){
        Hello h=new Hello(99);
        h.show();
    }
    class Hello {
        int a;
        Hello(){
            System.out.println("Hello Defl Cons");
        }
        Hello(int a){
            System.out.println("Hello 1-Arg Cons");
            this.a=a;
            this();
        }
        void show(){
            System.out.println(a);
        }
    }
}
```

OOPS80.java

```
class OOPS80{
    public static void main(String args[]){
        Hello h=new Hello(99);
        h.show();
    }
    class Hello {
        int a;
        Hello(){
            System.out.println("Hello Defl Cons");
        }
        Hello(int a){
            this();
            System.out.println("Hello 1-Arg Cons");
            this.a=a;
        }
        void show(){
            System.out.println(a);
        }
    }
}
```

```
System.out.println("Hello Defl Cons");
}
Hello(int a){
    this();
    System.out.println("Hello 1-Arg Cons");
    this.a=a;
}
void show(){
    System.out.println(a);
}
}
```

OOPS81.java

```
class OOPS81{
    public static void main(String args[]){
        Hello h=new Hello();
    }
    class Hello{
        Hello(){
            this();
        }
    }
}
```

OOPS82.java

```
class OOPS82{
    public static void main(String args[]){
        Hello h=new Hello();
    }
    class Hello {
        Hello () {
            this (10);
        }
        Hello (int a){
            this ();
        }
    }
}
```

OOPS83.java

```
class OOPS83{
    public static void main(String args[]){
        System.out.println("Main:"+Hello.a);
    }
    class Hello {
        static int a;
        static {
            String a="India"
            System.out.println("S. Block"+a);
            System.out.println("S. Block"+this.a);
        }
    }
}
```

OOPS84.java

```
class OOPS84{
    public static void main(String args[]){
        System.out.println("Main:"+Hello.a);
    }
    class Hello {
        static int a;
        static {
            String a="India";
            System.out.println("S. Block"+a);
            System.out.println("S. Block"+Hello.a);
        }
    }
}
```

OOPS85.java

```

class OOPS85{
    public static void main(String args[]){
        Student stu1=new
        Student(99,"Sai", "Sai@m",9999);
        stu1.show();
    }
}

class Student{
    int sid;
    String name;
    String email;
    long phone;

    Student() {
        System.out.println("Stud o-Arg Cons");
    }

    Student(int sid) {
        this.sid=sid;
        System.out.println("Stud 1-Arg Cons");
    }

    Student(int sid, String name) {
        System.out.println("Stud2-Arg Cons");
        this.sid=sid;
        this.name=name;
    }

    Student(int sid, String name, String email) {
        System.out.println("Stud 3-Arg Cons");
        this.sid=sid;
        this.name=name;
        this.email=email;
    }

    Student(int sid, String name, String email,
    long phone){
        System.out.println("Stud 4-Arg Cons");
        this.sid=sid;
        this.name=name;
        this.email=email;
        this.phone=phone;
    }

    void show() {
        System.out.println(sid);
        System.out.println(name);
        System.out.println(email);
        System.out.println(phone);
    }
}

```

OOPS86.java

```

class OOPS86{
    public static void main(String args[]){
        Student stu1=new
        Student(99,"Sai", "Sai@gmail",9999);
        stu1.show();
    }
}

class Student {
    int sid;
    String name;
    String email;
    long phone;

    Student() {
        System.out.println("Stud o-Arg Cons");
    }

    Student(int sid) {
        this();
        this.sid=sid;
        System.out.println("Stud 1-Arg Cons");
    }

    Student(int sid,String name) {
        this(sid);
        System.out.println("Stud-Arg Cons");
        this.name=name;
    }

    Student(int sid, String name, String email){
        this(sid, name);
        System.out.println("Stud 3-Arg Cons");
        this.email=email;
    }

    Student(int sid, String name, String email,
    long phone) {
        this(sid, name, email);
        System.out.println("Stud 4-Arg Cons");
        this.phone=phone;
    }

    void show(){
        System.out.println(sid);
        System.out.println(name);
        System.out.println(email);
        System.out.println(phone);
    }
}

```

OOPS87.java

```

class OOPS87{
    public static void main(String args[]){
        System.out.println("Hello");
    }
    class Hello{
        static {
            String a="India";
            System.out.println("S. Block"+a);
            System.out.println("S. Block"+Hello.a);
        }
    }
}

```

OOPS88.java

```

class OOPS88{
    public static void main(String args[]){
        System.out.println("Hello");
    }
    class Hello{
        {
            String a="India";
            System.out.println("I. Block"+a);
            System.out.println(this. a);
        }
    }
}

```

OOPS89.java

```

class OOPS89{
    public static void main(String args[]){
        Hello h=new Hello();
        h. show();
    }
    class Hello{
        void show(){
            this =null;
        }
    }
}

```

OOPS90.java

```

class OOPS90{
    public static void main(String args[]){
        Hello h=new Hello();
        h. show();
    }
    class Hello{
        void show(){
            this =new Hello();
        }
    }
}

```

OOPS91.java

```

class OOPS91{
    public static void main(String args[]){
        new Hello().show();
        new Hai().show();
    }
    class Hello{
        void show(){
            Hello h1=this;
            Hai h2=this;
        }
    }
    class Hai{
        void show(){
            Hello h1=this;
            Hai h2=this;
        }
    }
}

```

OOPS92.java

```

class OOPS92{
    public static void main(String args[]){
        Hello h=new Hello();
        h. show();
    }
    class Hello{
        int a;
        void show(){
            String a="India";
            System.out.println(a);
            System.out.println(this. a);
            System.out.println(Hello. this. a);
        }
    }
}

```

OOPS93.java

```

class OOPS93{
    public static void main(String args[]){
        Hello h=new Hello();
        h. show();
    }
    class Hello{
        int a;

```

```

        void show(){
            String a="India";
            System.out.println(a);
            System.out.println(this. a);
            System.out.println(Hai. this. a);
        }
    }
    class Hai {
        int a;
    }
}

```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS71	10	You are accessing variable directly then it will Check whether that variable is declared in the local scope or not. If found in the local scope, that local variable will be used.
OOPS72	India India	1 st variable a will refer local initialization.
OOPS73	India 10	Same as 604
OOPS74	India Sai 10 20	When you have same name for local variable and class level variables then. Refer the local variable directly and Refer the class level variable using this keyword.
OOPS75	Stud 2-Arg Cons 0 null Stud 2-Arg Cons 0 null	After compilation statement System.out.println (sid+ "\t"+sname); is converted into System.out.println (this.sid + "\t" + this. sname); so that it will refer class level variable.
OOPS76	Stud 2-Arg Cons 88 Ram Stud 2-Arg Cons 99 Sai	To access the actual value use this as reference.
OOPS77	Hello Def Cons 0 hello 1-arg Cons 99	Here, We are invoking one constructor from another constructor using this keyword.
OOPS78	Compilation error	[Error: The method Hello () is undefined for the type Hello], By overloading the constructor you cannot use class name but you can use this keyword.
OOPS79	Compilation error	[Error: Constructor this () must be the first statement in a constructor].
OOPS80	Hello Def Cons Hello 1-Arg Cons 99	By overloading the constructor you can use this keyword but it is 1 st statement of constructor
OOPS81	Compilation error	[Error: Recursive constructor invocation Hello ()]. Here, this () constructor refer to class and it will run as cyclic process.
OOPS82	Compilation error	[Error: Recursive constructor invocation Hello (int a), Recursive constructor invocation Hello ()].same as 611
OOPS83	Compilation error	[Error: non-static variable this cannot be referenced from a static context].
OOPS84	S. Block: India S. Block: 0 Main: 0	To refer static context use class name as reference.

Prog No.	Output	Error and Details
OOPS85	Stud 4-Arg Cons 99 Sai Sai@mail 9999	Here, you are invoking one constructor from another constructor using this keyword reference is called as constructor chaining.
OOPS86	Stud 0-Arg Cons Stud 1-Arg Cons Stud 2-Arg Cons Stud 3-Arg Cons Stud 4-Arg Cons 99 Sai Sai@gmail 9999	Here, we are using this as a constructor to call one constructor to another constructor.
OOPS87	Compilation error	[Error: cannot find symbol a].Here, Local variable cannot be referred using class name.
OOPS88	Compilation error	[Error: cannot find symbol a]. Because Local variable cannot be referred using “this” keyword.
OOPS89	Compilation error	[Error: The left-hand side of an assignment must be a variable]. Because, “this” is a final reference variable and cannot be modified.
OOPS90	Compilation error	[Error: The left-hand side of an assignment must be a variable]. Because, “this” is a final reference variable and cannot be modified.
OOPS91	Compilation error	[Error: Type mismatch: cannot convert from Hello to Hai]
OOPS92	India 0 0	Here, “a” is refer local variable and “this. a” and “Hello. this. a” will refer class level variable .
OOPS93	Compilation error	[Error: No enclosing instance of the type Hai is accessible in scope]we cannot use another class name as reference.

Write your note here.....